```
# Course: BSAN 360
# Name: Maxwell Davis
# Project Assignment: Part 6
# Date: March 15, 2026

# Project Topic:
# Finishing touches to add data visualization etc.
```

```
# Question 1
# Download and read the dataset into a DataFrame and verify no errors o

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

path = "/content/2023_nba_player_stats.csv"
df = pd.read_csv(path)

df.head()
```

	PName	POS	Team	Age	GP	W	L	Min	PTS	FGM	...	REB	AST	TOV	STL	BLK	PF	FP	DD2	TD3
0	Jayson Tatum	SF	BOS	25	74	52	22	2732.2	2225	727	...	649	342	213	78	51	160	3691	31	1
1	Joel Embiid	C	PHI	29	66	43	23	2284.1	2183	728	...	670	274	226	66	112	205	3706	39	1
2	Luka Doncic	PG	DAL	24	66	33	33	2390.5	2138	719	...	569	529	236	90	33	166	3747	36	10
3	Shai Gilgeous-Alexander	PG	OKC	24	68	33	35	2416.0	2135	704	...	329	371	192	112	65	192	3425	3	0
4	Giannis Antetokounmpo	PF	MIL	28	63	47	16	2023.6	1959	707	...	742	359	246	52	51	197	3451	46	6

5 rows × 30 columns

```
# Question 2
# Determine the size of the dataset (rows and columns)

df.shape
```

```
(539, 30)
```

```
# Question 3
# Inspect the first and last observations to verify correctness

df.head(5)
df.tail(5)
```

	PName	POS	Team	Age	GP	W	L	Min	PTS	FGM	...	REB	AST	TOV	STL	BLK	PF	FP	DD2	TD3	+/-
534	Alondes Williams	NaN	BKN	23	1	1	0	5.3	0	0	...	1	0	2	0	0	1	-1	0	0	-5
535	Deonte Burton	NaN	SAC	29	2	1	1	6.5	0	0	...	0	0	0	0	0	0	0	0	0	5
536	Frank Jackson	NaN	UTA	24	1	0	1	5.0	0	0	...	2	1	0	0	0	0	4	0	0	-2
537	Michael Foster Jr.	NaN	PHI	20	1	1	0	1.0	0	0	...	0	0	0	0	0	0	0	0	0	-2
538	Sterling Brown	NaN	LAL	28	4	2	2	24.4	0	0	...	8	2	0	3	0	4	22	0	0	-4

5 rows × 30 columns

```
# Question 4
# Verify column labels were read correctly

df.columns
```

```
Index(['PName', 'POS', 'Team', 'Age', 'GP', 'W', 'L', 'Min', 'PTS', 'FGM',
       'FGA', 'FG%', '3PM', '3PA', '3P%', 'FTM', 'FTA', 'FT%', 'OREB', 'DREB',
       'REB', 'AST', 'TOV', 'STL', 'BLK', 'PF', 'FP', 'DD2', 'TD3', '+/-'],
      dtype='object')
```

```
# Question 5
# Examine descriptive statistics and data quality

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 539 entries, 0 to 538
Data columns (total 30 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   PName   539 non-null    object
 1   POS     534 non-null    object
 2   Team    539 non-null    object
 3   Age     539 non-null    int64
 4   GP      539 non-null    int64
 5   W       539 non-null    int64
 6   L       539 non-null    int64
 7   Min     539 non-null    float64
 8   PTS     539 non-null    int64
 9   FGM     539 non-null    int64
 10  FGA     539 non-null    int64
 11  FG%     539 non-null    float64
 12  3PM     539 non-null    int64
 13  3PA     539 non-null    int64
 14  3P%     539 non-null    float64
 15  FTM     539 non-null    int64
 16  FTA     539 non-null    int64
 17  FT%     539 non-null    float64
 18  OREB    539 non-null    int64
 19  DREB    539 non-null    int64
 20  REB     539 non-null    int64
 21  AST     539 non-null    int64
 22  TOV     539 non-null    int64
 23  STL     539 non-null    int64
 24  BLK     539 non-null    int64
 25  PF      539 non-null    int64
 26  FP      539 non-null    int64
 27  DD2     539 non-null    int64
 28  TD3     539 non-null    int64
 29  +/-     539 non-null    int64
dtypes: float64(4), int64(23), object(3)
memory usage: 126.5+ KB
```

```
# Summary statistics for numeric columns
df.describe()
```

	Age	GP	W	L	Min	PTS	FGM	FGA	FG
count	539.000000	539.000000	539.000000	539.000000	539.000000	539.000000	539.000000	539.000000	539.00000
mean	25.970315	48.040816	24.018553	24.022263	1103.617625	523.426716	191.576994	403.005566	46.32523
std	4.315513	24.650686	14.496366	13.445866	827.765114	498.084360	178.351286	369.595909	10.96727
min	19.000000	1.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.00000
25%	23.000000	30.500000	12.000000	14.000000	329.000000	120.500000	45.500000	93.500000	41.65000
50%	25.000000	54.000000	25.000000	25.000000	970.200000	374.000000	138.000000	300.000000	45.50000
75%	29.000000	68.000000	36.000000	34.000000	1845.900000	769.500000	283.500000	598.500000	50.60000
max	42.000000	83.000000	57.000000	60.000000	2963.200000	2225.000000	728.000000	1559.000000	100.00000

8 rows × 27 columns

```
# Missing values check
df.isnull().sum()
```

	0
PName	0
POS	5
Team	0
Age	0
GP	0
W	0
L	0
Min	0

```
# Question 5C
# Check minimum and maximum values of key numeric columns

numeric_check = ["GP","Min","FGM","FGA","FG%","PTS","REB","AST","TOV","STL","BLK"]

df[numeric_check].agg(["min","max"])

# Range Interpretation:
# GP ranges between 0 and 82 which is realistic for an NBA season.
# FG% ranges between 0 and 1 which is correct for a percentage statistic.
# Points and attempts are within expected basketball values.
# No impossible values (like negative points or FG% > 1) were found.

# Check categorical columns for valid values

df["POS"].unique()

# Position categories appear valid (G, F, C combinations).
```

```
array(['SF', 'C', 'PG', 'PF', 'SG', 'F', 'G', nan], dtype=object)
```

```
# Question 5D
# Check for missing values in each column

df.isna().sum()
```

FP	0
DD2	0
TD3	0
+/-	0

dtype: int64

	0
PName	0
POS	5
Team	0
Age	0
GP	0
W	0
L	0
Min	0

```
# Missing Values Plan:
# Columns critical to my research question are FG%, FGA, PTS, and GP.
# Any rows missing these values will be removed because they prevent
# calculation of scoring and shooting efficiency.

# This is appropriate because imputing scoring or shooting data
# would distort the relationship I am studying.
```

3PA	0

```
# Column Information:
# PName = Player name (string)
# Team = Team abbreviation (string)
# GP = Games played (numeric)
# Min = Minutes played (numeric)
# FGM/FGA = Field goals made/attempted (numeric)
# FG% = Shooting efficiency (numeric percentage)
# PTS = Total points scored (numeric)

# Range Check:
# All numeric basketball statistics fall within realistic NBA ranges.
# FG% is between 0 and 1 which is correct.
```

```
# Missing Values Plan:
# If missing values exist in FG%, FGA, or PTS, those rows will be removed
# because they are essential to answering the research question.
```

```
PF    0
```

```
# Question 6 (UPDATED)
# Restate research questions + determine needed columns (including position big or small + free throws)

# Research Questions:
# 1) How does FG% relate to scoring (PTS / PPG)?
# 2) Can lower FG% players still score more because they take more shots (FGA)?
# Teacher feedback updates:
# - Filter out players with very few minutes/zeros using thresholds (handled in Q7)
# - Position matters: simplify POS into BIG vs SMALL (handled in Q7/Q8)
# - Add free throws since they contribute to points (FTM/FTA/FT%)

needed_columns = ["PName","Team","POS","GP","Min","PTS","FGM","FGA","FG%","FTM","FTA","FT%"]
df[needed_columns].head()
```

	PName	Team	POS	GP	Min	PTS	FGM	FGA	FG%	FTM	FTA	FT%
0	Jayson Tatum	BOS	SF	74	2732.2	2225	727	1559	46.6	531	622	85.4
1	Joel Embiid	PHI	C	66	2284.1	2183	728	1328	54.8	661	771	85.7
2	Luka Doncic	DAL	PG	66	2390.5	2138	719	1449	49.6	515	694	74.2
3	Shai Gilgeous-Alexander	OKC	PG	68	2416.0	2135	704	1381	51.0	669	739	90.5
4	Giannis Antetokounmpo	MIL	PF	63	2023.6	1959	707	1278	55.3	498	772	64.5

```
# Question 7 (UPDATED)
# Create a focused project dataset + remove low-minute/low-scoring players
```

```
# Also create SMALL vs BIG (0/1) position variable for easier modeling

import numpy as np
import pandas as pd

project_df = df[needed_columns].copy()

# Ensure numeric columns are numeric (avoid hidden strings causing issues)
num_cols = ["GP","Min","PTS","FGM","FGA","FG%","FTM","FTA","FT%"]
for c in num_cols:
    project_df[c] = pd.to_numeric(project_df[c], errors="coerce")

# Drop rows missing the core values needed for your analysis
project_df = project_df.dropna(subset=["POS","GP","Min","PTS","FGA","FG%","FTM","FTA","FT%"]).reset_index(drop

#Threshold filters (teacher feedback)
# Eyeballed what a reasonable minimum minute threshold would be as a baskwtball player myself and considering
MIN_MINUTES = 400
MIN_POINTS  = 100

project_df = project_df[(project_df["Min"] >= MIN_MINUTES) & (project_df["PTS"] >= MIN_POINTS)].reset_index(dr

# Create SMALL vs BIG
# Basketball note: guards = backcourt (small), forwards/centers = frontcourt (big)
# SMALL = 0, BIG = 1
pos = project_df["POS"].astype(str).str.upper().str.strip()

# BIG if contains C OR contains F (including PF/SF, etc.)
# SMALL if primarily guard-type
project_df["BIG"] = np.where(pos.str.contains("C") | pos.str.contains("F"), 1, 0).astype(int)

# Quick check
print("Rows after filtering:", project_df.shape[0])
project_df[["PName","Team","POS","BIG","Min","PTS","FGA","FG%","FTM","FTA"]].head(10)
```

```
Rows after filtering: 388
```

	PName	Team	POS	BIG	Min	PTS	FGA	FG%	FTM	FTA
0	Jayson Tatum	BOS	SF	1	2732.2	2225	1559	46.6	531	622
1	Joel Embiid	PHI	C	1	2284.1	2183	1328	54.8	661	771
2	Luka Doncic	DAL	PG	0	2390.5	2138	1449	49.6	515	694
3	Shai Gilgeous-Alexander	OKC	PG	0	2416.0	2135	1381	51.0	669	739
4	Giannis Antetokounmpo	MIL	PF	1	2023.6	1959	1278	55.3	498	772
5	Anthony Edwards	MIN	SG	0	2841.5	1946	1541	45.9	319	422
6	Julius Randle	NYK	PF	1	2737.3	1936	1432	45.9	402	531
7	Donovan Mitchell	CLE	SG	0	2432.4	1922	1402	48.4	319	368
8	Trae Young	ATL	PG	0	2540.7	1914	1390	42.9	566	639
9	Zach LaVine	CHI	SG	0	2767.9	1913	1388	48.5	363	428

```
# Proj. assignment 3 - First data manipulation!! Processing Strings.
# Merging was not needed due to only having one file.

# explanation:

# Clean and standardize text columns (name, team, position)
# Removes spacing/symbol issues and makes categories consistent
# Splits player name into first and last name for easier analysis
# Clean/standardize key text columns


project_df["PName"] = (
    project_df["PName"]
    .astype(str)
```

```
    .str.replace("*", "", regex=False)   # common cleaning step if names include *
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

project_df["Team"] = project_df["Team"].astype(str).str.upper().str.strip()
project_df["POS"]  = project_df["POS"].astype(str).str.upper().str.strip()

# Example "parse strings" step (split player name)
# Creates two new columns: FirstName and LastName
name_parts = project_df["PName"].str.split(" ", n=1, expand=True)
project_df["FirstName"] = name_parts[0]
project_df["LastName"]  = name_parts[1].fillna("")  # handles 1-word names safely

project_df[["PName","FirstName","LastName","Team","POS"]].head(10)
```

	PName	FirstName	LastName	Team	POS
0	Jayson Tatum	Jayson	Tatum	BOS	SF
1	Joel Embiid	Joel	Embiid	PHI	C
2	Luka Doncic	Luka	Doncic	DAL	PG
3	Shai Gilgeous-Alexander	Shai	Gilgeous-Alexander	OKC	PG
4	Giannis Antetokounmpo	Giannis	Antetokounmpo	MIL	PF
5	Anthony Edwards	Anthony	Edwards	MIN	SG
6	Julius Randle	Julius	Randle	NYK	PF
7	Donovan Mitchell	Donovan	Mitchell	CLE	SG
8	Trae Young	Trae	Young	ATL	PG
9	Zach LaVine	Zach	LaVine	CHI	SG

```
# Quick check to confirm the exact column names in project_df (error came up)
print(project_df.columns.tolist())
```

```
['PName', 'Team', 'POS', 'GP', 'Min', 'PTS', 'FGM', 'FGA', 'FG%', 'FTM', 'FTA', 'FT%', 'BIG', 'FirstName', 'Las
```

```
# Proj assignment 3 - More data cleaning/meanipulation - Reshaping & Pivoting

# Explanation:

# Reorganizes data to compare BIG vs SMALL players by team
# Pivot shows average PPG for each group
# Melt converts it back to long format for analysis


# Find the correct "points per game" column name
possible_ppg_cols = ["PPG", "PTS", "PTS/G", "Points", "PointsPerGame"]

ppg_col = None
for c in possible_ppg_cols:
    if c in project_df.columns:
        ppg_col = c
        break

if ppg_col is None:
    raise KeyError(
        "Could not find a points-per-game column. "
        "Check project_df.columns.tolist() and update possible_ppg_cols."
    )

print("Using points column:", ppg_col)

# Pivot: average points per game by Team and BIG
ppg_pivot = project_df.pivot_table(
    index="Team",
    columns="BIG",
    values=ppg_col,
    aggfunc="mean"
)
```

```
ppg_pivot = ppg_pivot.rename(columns={0: "Avg_PPG_SMALL", 1: "Avg_PPG_BIG"}).reset_index()
display(ppg_pivot.head(10))

# Melt back to long format
ppg_long = pd.melt(
    ppg_pivot,
    id_vars="Team",
    value_vars=["Avg_PPG_SMALL", "Avg_PPG_BIG"],
    var_name="PlayerGroup",
    value_name="Avg_PPG"
)

display(ppg_long.head(10))
```

```
Using points column: PTS
```

	BIG	Team	Avg_PPG_SMALL	Avg_PPG_BIG
0		ATL	937.800000	722.375000
1		BKN	574.833333	701.875000
2		BOS	954.800000	635.875000
3		CHA	570.875000	535.833333
4		CHI	820.800000	829.833333
5		CLE	927.400000	656.500000
6		DAL	972.400000	547.285714
7		DEN	793.250000	732.333333
8		DET	539.375000	575.714286
9		GSW	1020.166667	545.833333

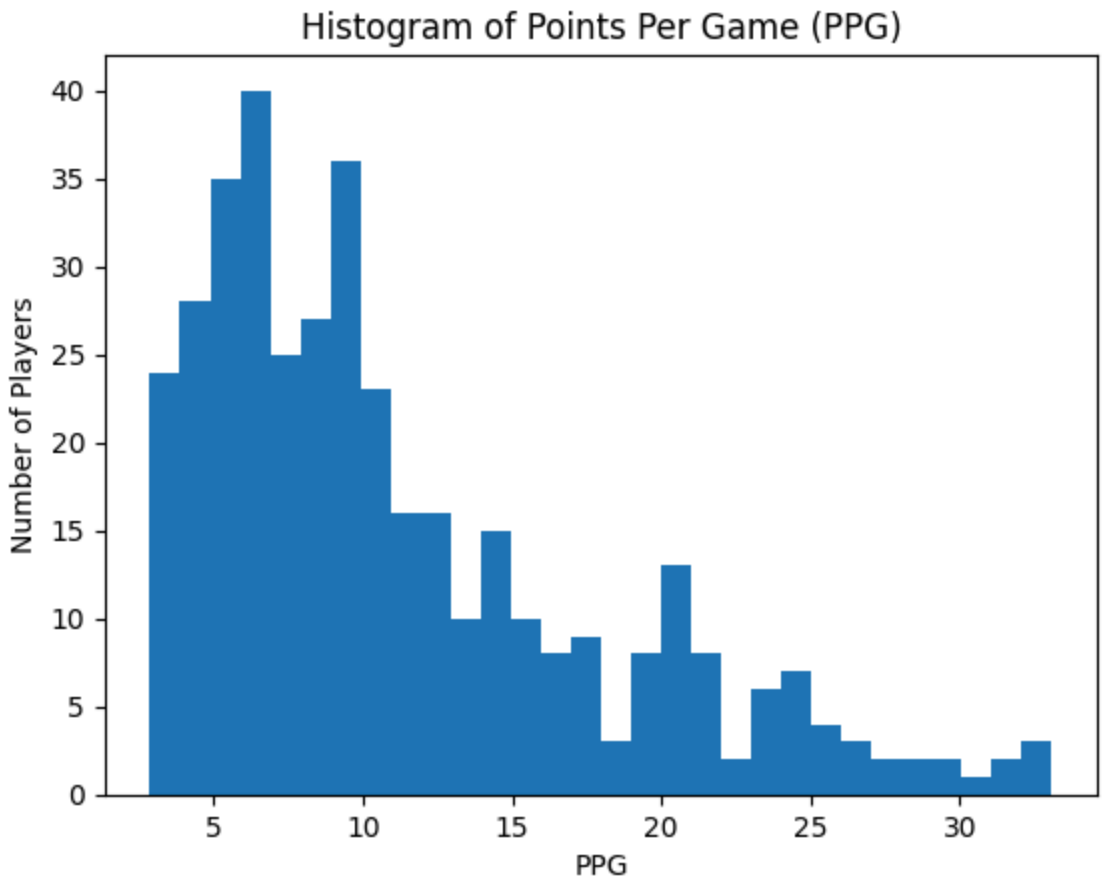
	Team	PlayerGroup	Avg_PPG
0	ATL	Avg_PPG_SMALL	937.800000
1	BKN	Avg_PPG_SMALL	574.833333
2	BOS	Avg_PPG_SMALL	954.800000
3	CHA	Avg_PPG_SMALL	570.875000
4	CHI	Avg_PPG_SMALL	820.800000
5	CLE	Avg_PPG_SMALL	927.400000
6	DAL	Avg_PPG_SMALL	972.400000
7	DEN	Avg_PPG_SMALL	793.250000
8	DET	Avg_PPG_SMALL	539.375000
9	GSW	Avg_PPG_SMALL	1020.166667

```
# Question 8 (UPDATED)
# Determine analyses and compute features for correlations and modeling
# Includes grouping by BIG vs SMALL and adds free throws to the model

import statsmodels.api as sm

# Create per-game metrics - better comparison stat
project_df["PPG"] = project_df["PTS"] / project_df["GP"]
project_df["FGA_PG"] = project_df["FGA"] / project_df["GP"]
project_df["FTA_PG"] = project_df["FTA"] / project_df["GP"]
project_df["FTM_PG"] = project_df["FTM"] / project_df["GP"]
project_df["MIN_PG"] = project_df["Min"] / project_df["GP"]

# 1) Overall correlations
project_df[["PPG","FG%","FGA_PG","FTA_PG","MIN_PG","BIG"]].corr(numeric_only=True)
```

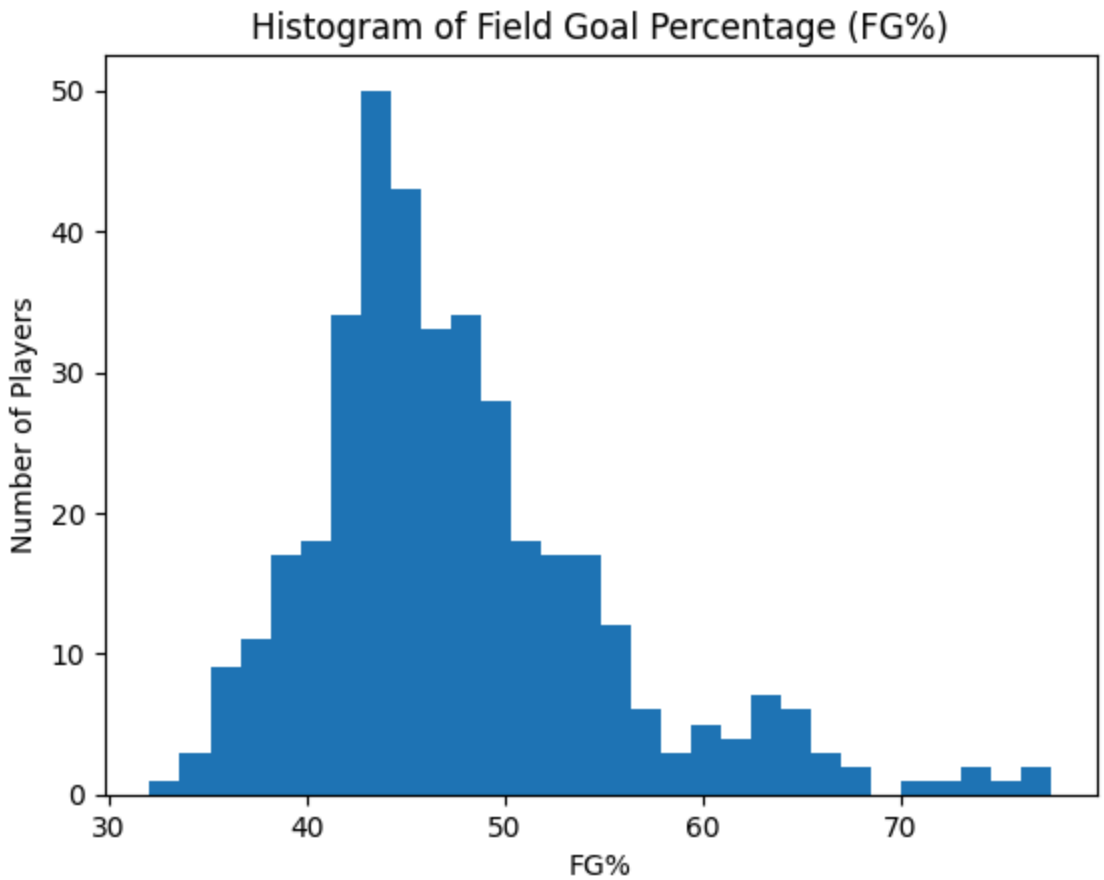
	PPG	FG%	FGA_PG	FTA_PG	MIN_PG	BIG
PPG	1.000000	0.089294	0.978288	0.889106	0.860760	-0.126764
FG%	0.089294	1.000000	-0.046072	0.191517	0.049172	0.419842
FGA_PG	0.978288	-0.046072	1.000000	0.813471	0.869959	-0.198418
FTA_PG	0.889106	0.191517	0.813471	1.000000	0.691558	-0.014611
MIN_PG	0.860760	0.049172	0.869959	0.691558	1.000000	-0.122646
BIG	-0.126764	0.419842	-0.198418	-0.014611	-0.122646	1.000000

```
# Plot 1: Histogram of PPG
plt.hist(project_df["PPG"], bins=30)
plt.title("Histogram of Points Per Game (PPG)")
plt.xlabel("PPG")
plt.ylabel("Number of Players")
```

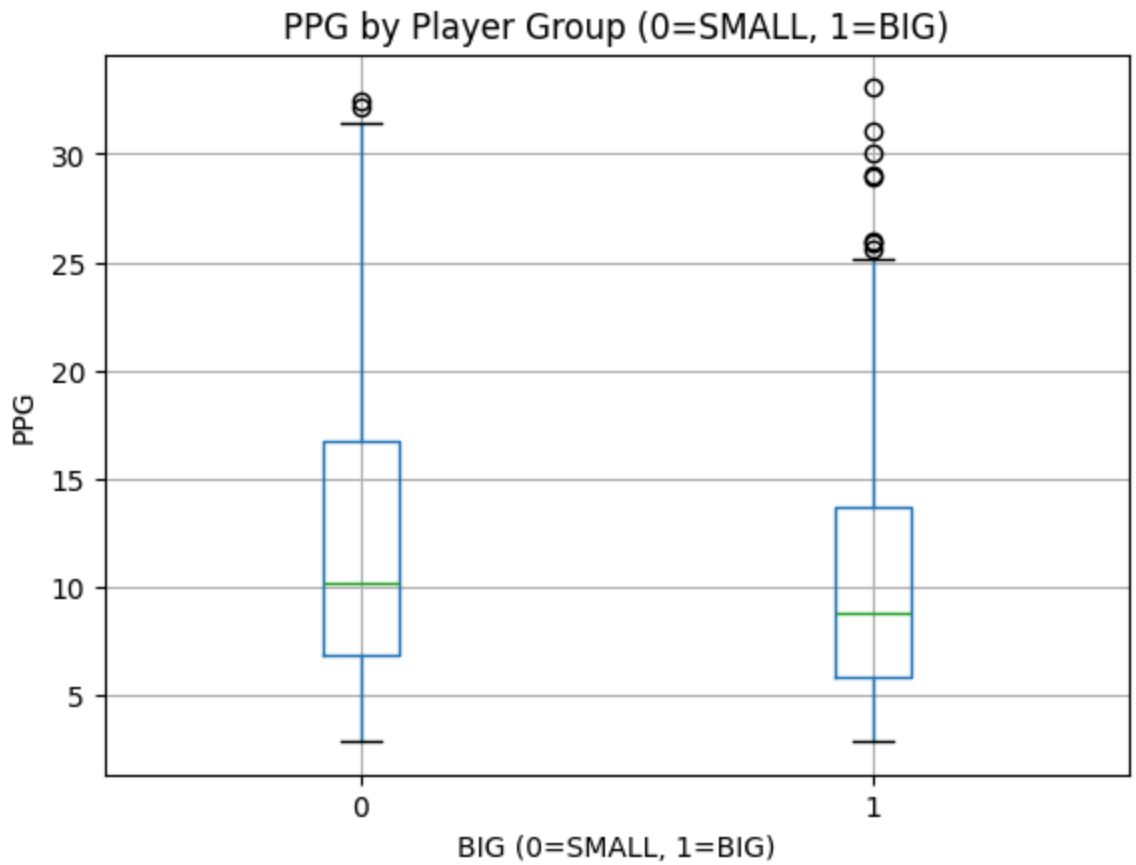
```
plt.show()

# Helps show typical scoring levels and whether there are outliers/high scorers.
```



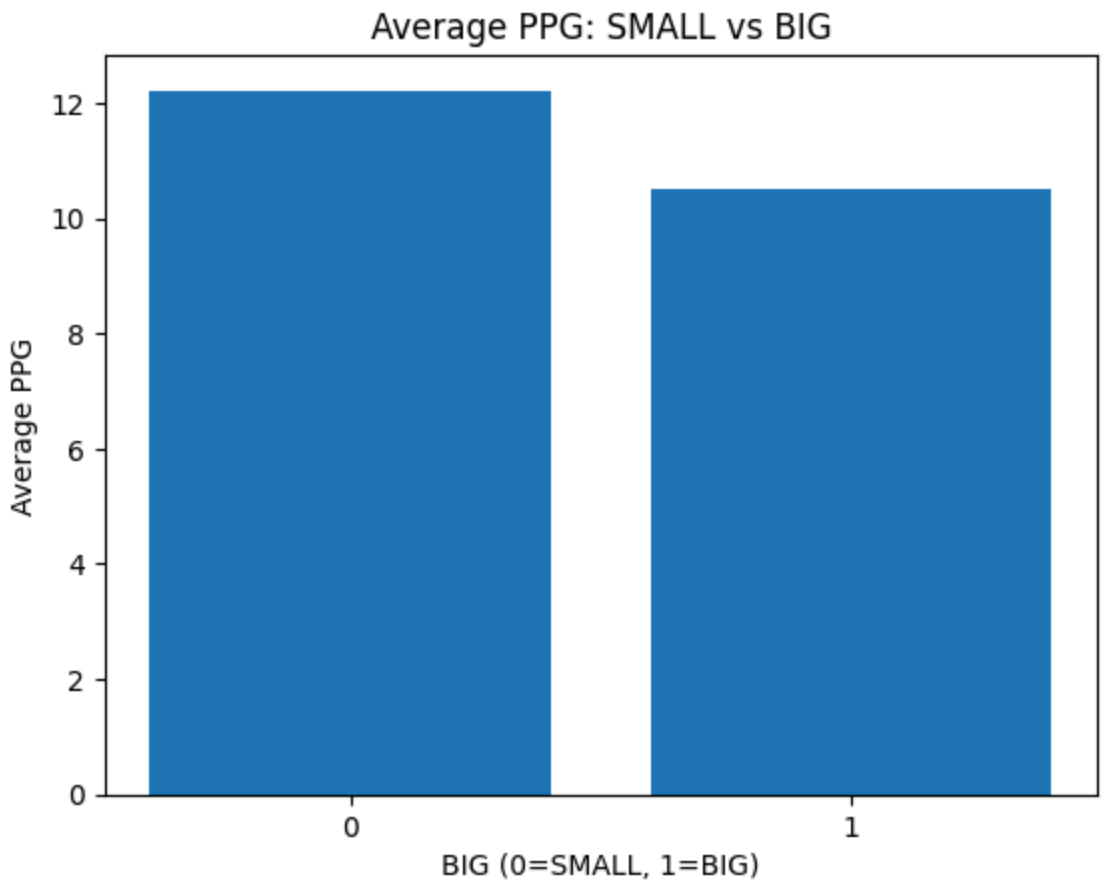

```
# Plot 2: Histogram of FG%
plt.hist(project_df["FG%"], bins=30)
plt.title("Histogram of Field Goal Percentage (FG%)")
plt.xlabel("FG%")
plt.ylabel("Number of Players")
plt.show()

# Shows the spread of shooting efficiency and if most players cluster around a range.
```




```
# Plot 3: Boxplot of PPG grouped by BIG vs SMALL
# BIG: 0 = SMALL, 1 = BIG

project_df.boxplot(column="PPG", by="BIG")
plt.title("PPG by Player Group (0=SMALL, 1=BIG)")
plt.suptitle("")  # removes automatic subtitle
plt.xlabel("BIG (0=SMALL, 1=BIG)")
plt.ylabel("PPG")
plt.show()

# Boxplot to show median, spread, and outliers. In order to compare distributions.
```

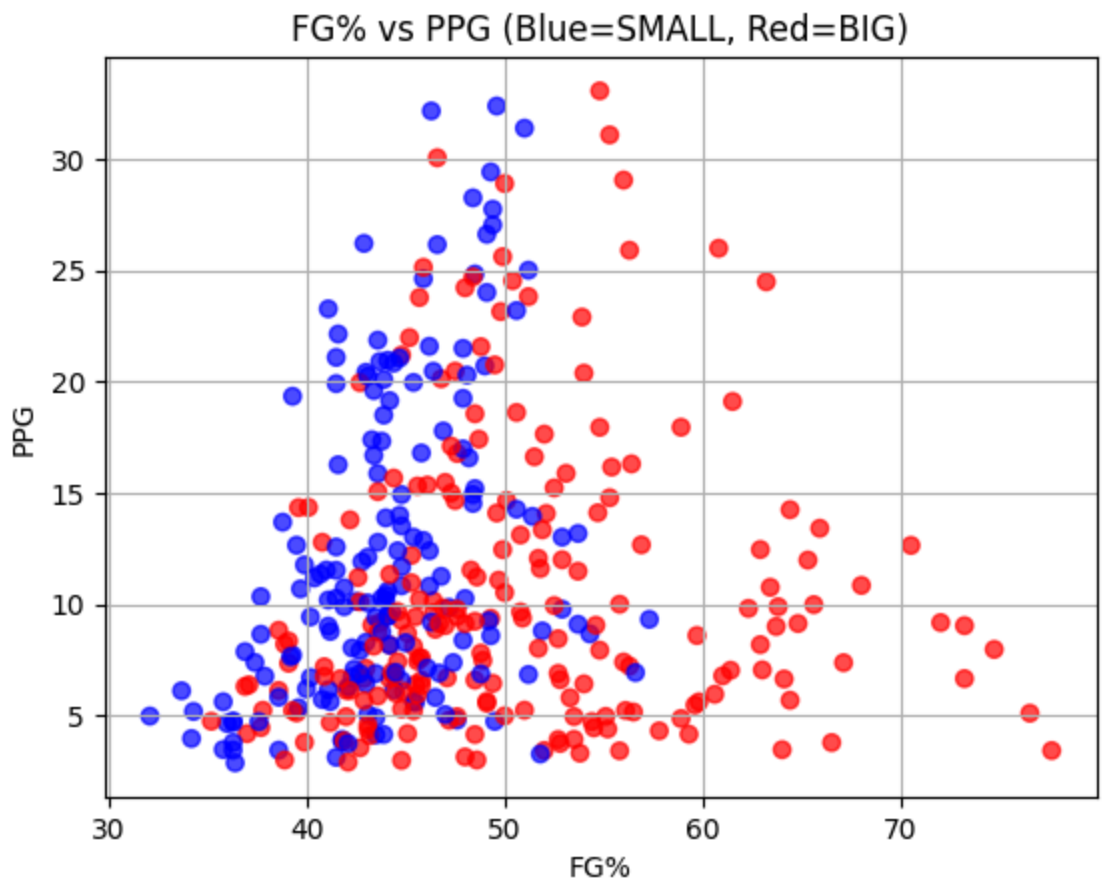



```
# Plot 4: Bar chart of average PPG for BIG vs SMALL
avg_ppg_by_group = project_df.groupby("BIG")["PPG"].mean().reset_index()

plt.bar(avg_ppg_by_group["BIG"].astype(str), avg_ppg_by_group["PPG"])
plt.title("Average PPG: SMALL vs BIG")
plt.xlabel("BIG (0=SMALL, 1=BIG)")
plt.ylabel("Average PPG")
plt.show()

# Summarizes whether one group scores more on average.
```



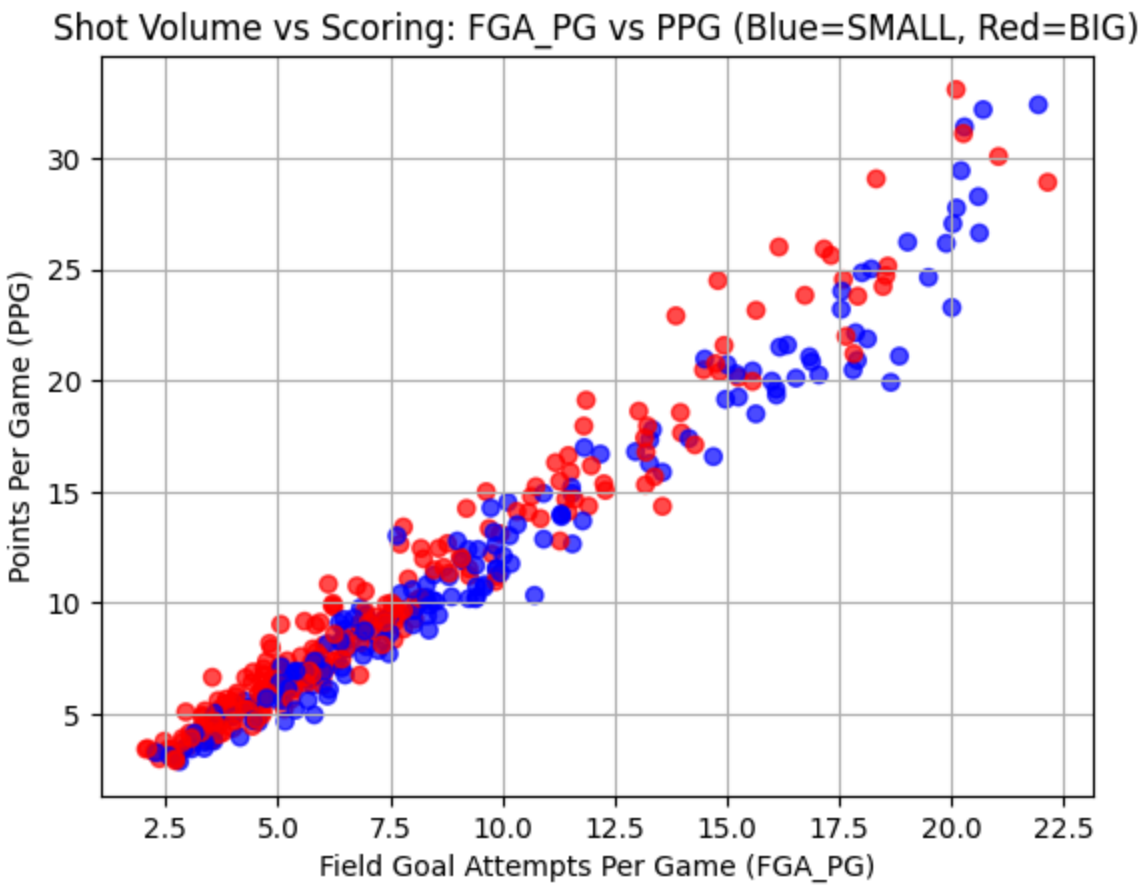

```
# Plot 5: Scatterplot of FG% vs PPG (colored by BIG)

colors = project_df["BIG"].copy()
colors = colors.replace({0: "blue", 1: "red"})

plt.scatter(project_df["FG%"], project_df["PPG"], c=colors, alpha=0.7)
plt.title("FG% vs PPG (Blue=SMALL, Red=BIG)")
plt.xlabel("FG%")
plt.ylabel("PPG")
plt.grid(True)
plt.show()
```

```
# Whether high scorers tend to be more/less efficient
# and whether BIG/SMALL groups show different patterns.
```

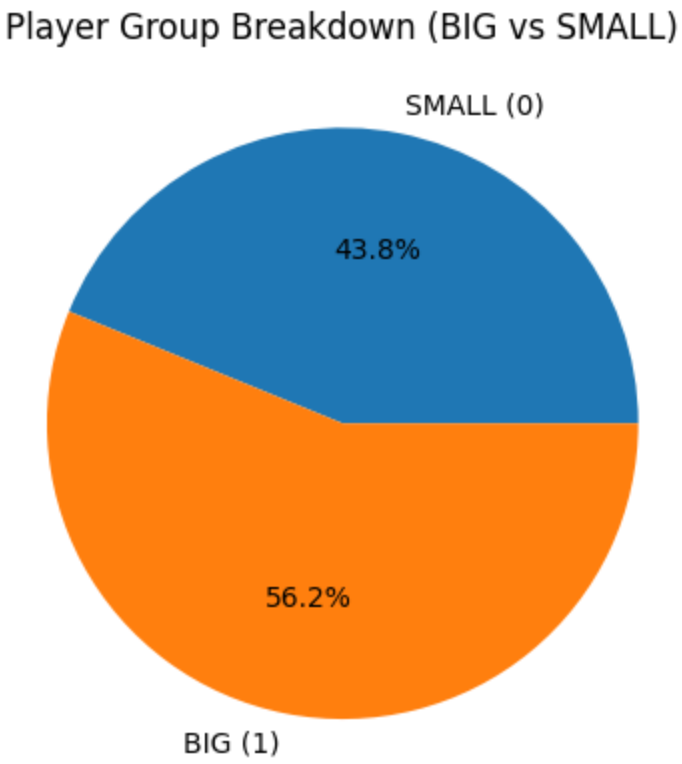



```
# Plot 6: Scatterplot of FGA per Game vs PPG (shot volume vs scoring)

colors = project_df["BIG"].copy()
colors = colors.replace({0: "blue", 1: "red"})

plt.scatter(project_df["FGA_PG"], project_df["PPG"], c=colors, alpha=0.7)
plt.title("Shot Volume vs Scoring: FGA_PG vs PPG (Blue=SMALL, Red=BIG)")
plt.xlabel("Field Goal Attempts Per Game (FGA_PG)")
plt.ylabel("Points Per Game (PPG)")
plt.grid(True)
```

```
plt.show()

# Whether taking more shots is linked to scoring more...
# even if FG% is not high.
```




```
# Pie chart of BIG vs SMALL player counts just to show sample composition
counts = project_df["BIG"].value_counts().sort_index()  # 0 then 1

plt.pie(counts, labels=["SMALL (0)", "BIG (1)"], autopct="%1.1f%%")
plt.title("Player Group Breakdown (BIG vs SMALL)")
plt.show()
```




```
# Planned Analysis (UPDATED based on teacher feedback):
# 1) Use correlations to see how FG% relates to scoring (PPG), and how shot volume (FGA_PG) relates to scoring
# 2) Group by SMALL vs BIG (0/1) to see if relationships differ by player type.
# 3) Use regression to predict scoring (PPG) with FG% + shot attempts + free throws + minutes + BIG.

# 2) Correlations by SMALL vs BIG
corr_by_big = project_df.groupby("BIG")[["PPG","FG%","FGA_PG","FTA_PG","MIN_PG"]].corr(numeric_only=True)
corr_by_big
```

BIG		PPG	FG%	FGA_PG	FTA_PG	MIN_PG
0	PPG	1.000000	0.375659	0.982487	0.881825	0.873020
	FG%	0.375659	1.000000	0.284670	0.316848	0.359871
	FGA_PG	0.982487	0.284670	1.000000	0.822841	0.880653
	FTA_PG	0.881825	0.316848	0.822841	1.000000	0.704400
	MIN_PG	0.873020	0.359871	0.880653	0.704400	1.000000
1	PPG	1.000000	0.068957	0.978795	0.907283	0.848420
	FG%	0.068957	1.000000	-0.072884	0.187341	0.012858
	FGA_PG	0.978795	-0.072884	1.000000	0.834762	0.863688
	FTA_PG	0.907283	0.187341	0.834762	1.000000	0.687983
	MIN_PG	0.848420	0.012858	0.863688	0.687983	1.000000

Next steps: Generate code with corr_by_big New interactive sheet

```
# Question 9: Group the data by BIG and compare summary statistics

grouped_big = project_df.groupby("BIG")

print("Number of players in each BIG group:")
print(grouped_big.size())

print("\nAverage values by BIG group:")
print(grouped_big[["PPG", "FG%", "FGA_PG", "FTA_PG", "MIN_PG"]].mean())
```

```
Number of players in each BIG group:
BIG
0    170
1    218
dtype: int64
```

```
Average values by BIG group:
           PPG        FG%    FGA_PG    FTA_PG     MIN_PG
BIG
0    12.215184  44.075294  9.754768  2.349217  24.680460
1    10.497190  50.509633  7.816655  2.290685  22.790137
```

```
# Question 10: Show the top 5 scorers in each BIG group using apply()

def top_scorers(group):
    return group.sort_values("PPG", ascending=False).head(5)

top_players_by_group = grouped_big.apply(top_scorers)

top_players_by_group[["PName", "Team", "POS", "BIG", "PPG", "FG%", "FGA_PG", "FTA_PG"]]
```

```
/tmp/ipykernel_358/3692964193.py:6: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns
  top_players_by_group = grouped_big.apply(top_scorers)
```

BIG		PName	Team	POS	BIG	PPG	FG%	FGA_PG	FTA_PG
0	**2**	Luka Doncic	DAL	PG	0	32.393939	49.6	21.954545	10.515152
	10	Damian Lillard	POR	PG	0	32.172414	46.3	20.724138	9.620690
	3	Shai Gilgeous-Alexander	OKC	PG	0	31.397059	51.0	20.308824	10.867647
	20	Stephen Curry	GSW	PG	0	29.428571	49.3	20.232143	5.017857
	7	Donovan Mitchell	CLE	SG	0	28.264706	48.4	20.617647	5.411765
1	**1**	Joel Embiid	PHI	C	1	33.075758	54.8	20.121212	11.681818
	4	Giannis Antetokounmpo	MIL	PF	1	31.095238	55.3	20.285714	12.253968
	0	Jayson Tatum	BOS	SF	1	30.067568	46.6	21.067568	8.405405
	41	Kevin Durant	PHX	PF	1	29.063830	56.0	18.340426	7.106383
	24	LeBron James	LAL	SF	1	28.909091	50.0	22.163636	5.945455

```
# Question 11: Build a pivot table of average PPG by Team and BIG

ppg_team_big = project_df.pivot_table(
    values="PPG",
    index="Team",
    columns="BIG",
    aggfunc="mean"
)

ppg_team_big = ppg_team_big.rename(columns={0: "Avg_PPG_SMALL", 1: "Avg_PPG_BIG"})
```

```
ppg_team_big.head(10)
```

BIG Team	Avg_PPG_SMALL	Avg_PPG_BIG
ATL	13.874409	10.322018
BKN	9.555496	10.413297
BOS	14.220854	9.542377
CHA	12.251039	9.120999
CHI	10.973987	11.406878
CLE	14.086636	9.253730
DAL	16.033256	8.544204
DEN	11.428260	11.406060
DET	10.958270	11.093492
GSW	15.466992	9.277307

Next steps: Generate code with ppg_team_big New interactive sheet

```
# Question 12: Build a pivot table of average FG% by POS and BIG

fg_pos_big = project_df.pivot_table(
    values="FG%",
    index="POS",
    columns="BIG",
    aggfunc="mean"
)

fg_pos_big = fg_pos_big.rename(columns={0: "Avg_FG_SMALL", 1: "Avg_FG_BIG"})

fg_pos_big
```

BIG POS	Avg_FG_SMALL	Avg_FG_BIG
C	NaN	58.918182
F	NaN	46.365625
G	42.090909	NaN
PF	NaN	49.454687
PG	44.175758	NaN
SF	NaN	46.594030
SG	44.526829	NaN

Next steps: Generate code with fg_pos_big New interactive sheet

```
# Question 13: Build a crosstab of POS by BIG

pos_big_ct = pd.crosstab(project_df["POS"], project_df["BIG"])

pos_big_ct
```

BIG	0	1
POS		
C	0	55
F	0	32
G	22	0
PF	0	64
PG	66	0
SF	0	67
SG	82	0

Next steps: Generate code with pos_big_ct New interactive sheet

```
# Question 14: Build a crosstab of Team by BIG with totals

team_big_ct = pd.crosstab(project_df["Team"], project_df["BIG"], margins=True)

team_big_ct
```

BIG	0	1	All
Team			
ATL	5	8	13
BKN	6	8	14
BOS	5	8	13
CHA	8	6	14
CHI	5	6	11
CLE	5	6	11
DAL	5	7	12
DEN	4	9	13

Next steps: Generate code with team_big_ct New interactive sheet

```
# Assignment 6 Advanced Analysis
# Exploring scoring efficiency and shot volume.

# 1. Create a scoring efficiency metric (points per shot attempt).
# 2. Compare efficiency between BIG and SMALL players.
# 3. Compare efficiency between high-volume and low-volume shooters.
```

```
# Analysis 1: Create Efficiency Metric

# Points per Shot Attempt (efficiency metric)
# Uses total points divided by total field goal attempts

project_df["PTS_per_Shot"] = project_df["PTS"] / project_df["FGA"]

# Check results
project_df[["PName","PTS","FGA","PTS_per_Shot"]].head()
```

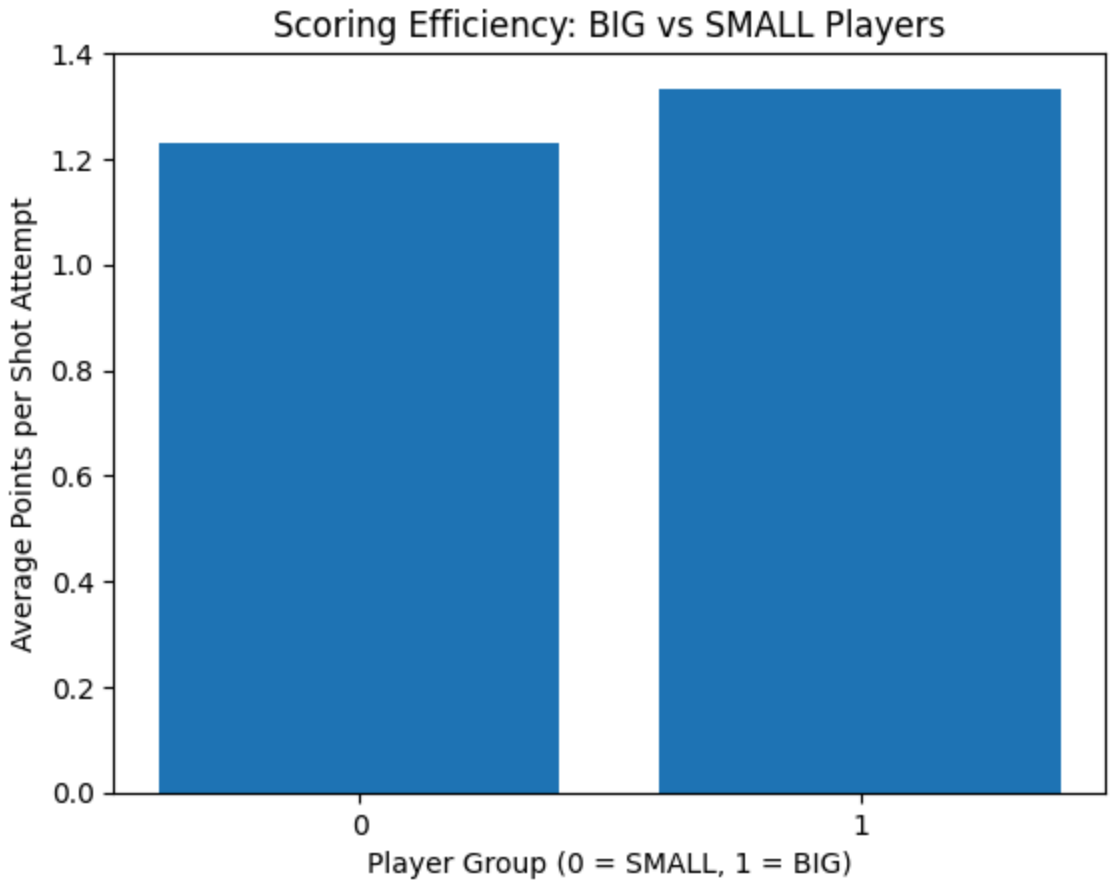
NOP	5	8	13
NYK	7	6	13
OKC	6	8	14

ORL	6	8	14
PHI	4	9	13
POR	5	9	14
SAS	6	9	15

	PName	PTS	FGA	PTS_per_Shot
0	Jayson Tatum	2225	1559	1.427197
1	Joel Embiid	2183	1328	1.643825
2	Luka Doncic	2138	1449	1.475500
3	Shai Gilgeous-Alexander	2135	1381	1.545981
4	Giannis Antetokounmpo	1959	1278	1.532864

```
# Analysis 2: Efficiency Comparison (BIG vs SMALL)

# Group by BIG vs SMALL
efficiency_by_size = project_df.groupby("BIG")["PTS_per_Shot"].mean()

print("Average Points per Shot Attempt (Efficiency):")
print(efficiency_by_size)

# Visualization
plt.bar(efficiency_by_size.index.astype(str), efficiency_by_size.values)

plt.title("Scoring Efficiency: BIG vs SMALL Players")
plt.xlabel("Player Group (0 = SMALL, 1 = BIG)")
plt.ylabel("Average Points per Shot Attempt")

plt.show()
```

```
Average Points per Shot Attempt (Efficiency):
BIG
0    1.229356
1    1.333395
Name: PTS_per_Shot, dtype: float64
```

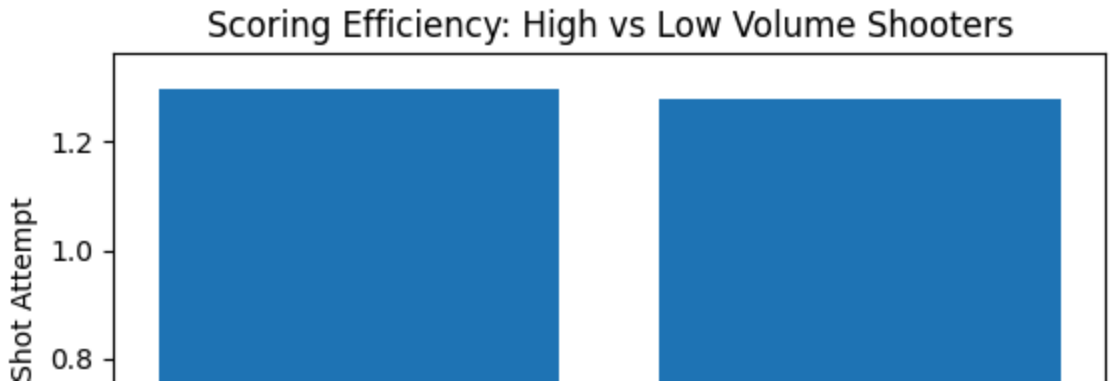



```
# Analysis 3: Create High vs Low Volume Shooters


# Determine the median shot attempts per game
median_fga = project_df["FGA_PG"].median()

# Create new volume group variable
project_df["Volume_Group"] = np.where(
```

```
    project_df["FGA_PG"] >= median_fga,
    "High Volume",
    "Low Volume"
)

# Check counts
project_df["Volume_Group"].value_counts()
```

Volume_Group	count
High Volume	194
Low Volume	194

dtype: int64

```
# Analysis 4: Efficiency Comparison (Shot Volume)

efficiency_by_volume = project_df.groupby("Volume_Group")["PTS_per_Shot

print("Average Points per Shot Attempt by Volume Group:")
print(efficiency_by_volume)

# Visualization
plt.bar(efficiency_by_volume.index, efficiency_by_volume.values)

plt.title("Scoring Efficiency: High vs Low Volume Shooters")
plt.xlabel("Shot Volume Group")
plt.ylabel("Average Points per Shot Attempt")

plt.show()
```

```
Average Points per Shot Attempt by Volume Group:
Volume_Group
High Volume    1.298094
Low Volume     1.277528
Name: PTS_per_Shot, dtype: float64
```